Web Programming

Javascript

Agenda

- JS 개요
- JS – 기본 연산자
- 객체 & 배열
- 함수
- 조건문 & 반복문

JS 개요

JS – 기본 연산자
객체 & 배열
함수
조건문 & 반복문

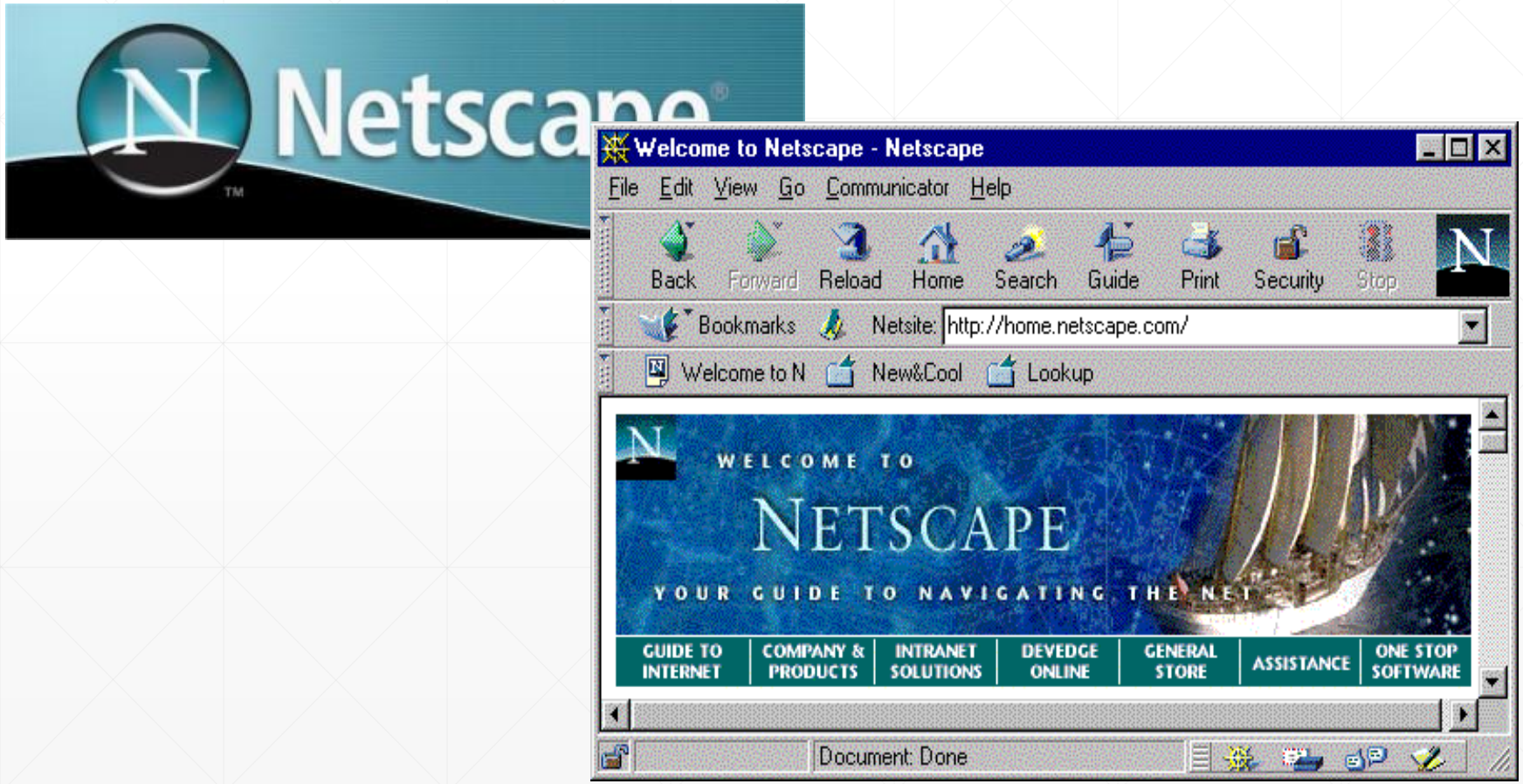
자바스크립트: 역사 (1/7)

- 웹 페이지 조작을 위한 스크립트 언어
  - HTML: 웹 페이지의 기본 구조
  - CSS: 디자인과 스타일 적용
  - Javascript: 웹 페이지의 동작 및 동적 조작

- Java? Javascript?
  - Java와는 별 관계가 없다!
  - 인도와 인도네시아의 관계

자바스크립트: 역사 (2/7)

- Mocha → LiveScript → Javascript
  - 넷스케이프의 어플리케이션 개발용 언어에서 진화



자바스크립트: 역사 (3/7)

- 음…자바… 음… 웹…

Marc Andreessen
(Founder of Netscape)

자바 같은데 자바 같지 않은,

그러면서도 간결하고 쉬운,

웹에 다이나믹함을 추가할 수 있는…
그런 언어 좀 만들어올래?

자바스크립트: 역사 (4/7)

- Mocha
  - 자바스크립트의 첫 번째 버전

10… 10일만
기다려줄래? ,,><,,

Brendan Eich

자바스크립트: 역사 (5/7)

LiveScript?
“Live” 멋진듯.

자바가 뜬댄다.
자바 붙여라

Ok!

JavaScript

자바스크립트: 역사 (6/7)

- 1995년, 자바스크립트가 Netscape에 탑재되기 시작
  - 새로운 사용자 경험과 함께 웹 시장 점유 시작

Microsoft

저거, 좋아 보인다.
우리도, 만든다.

자바스크립트: 역사 (6/7)

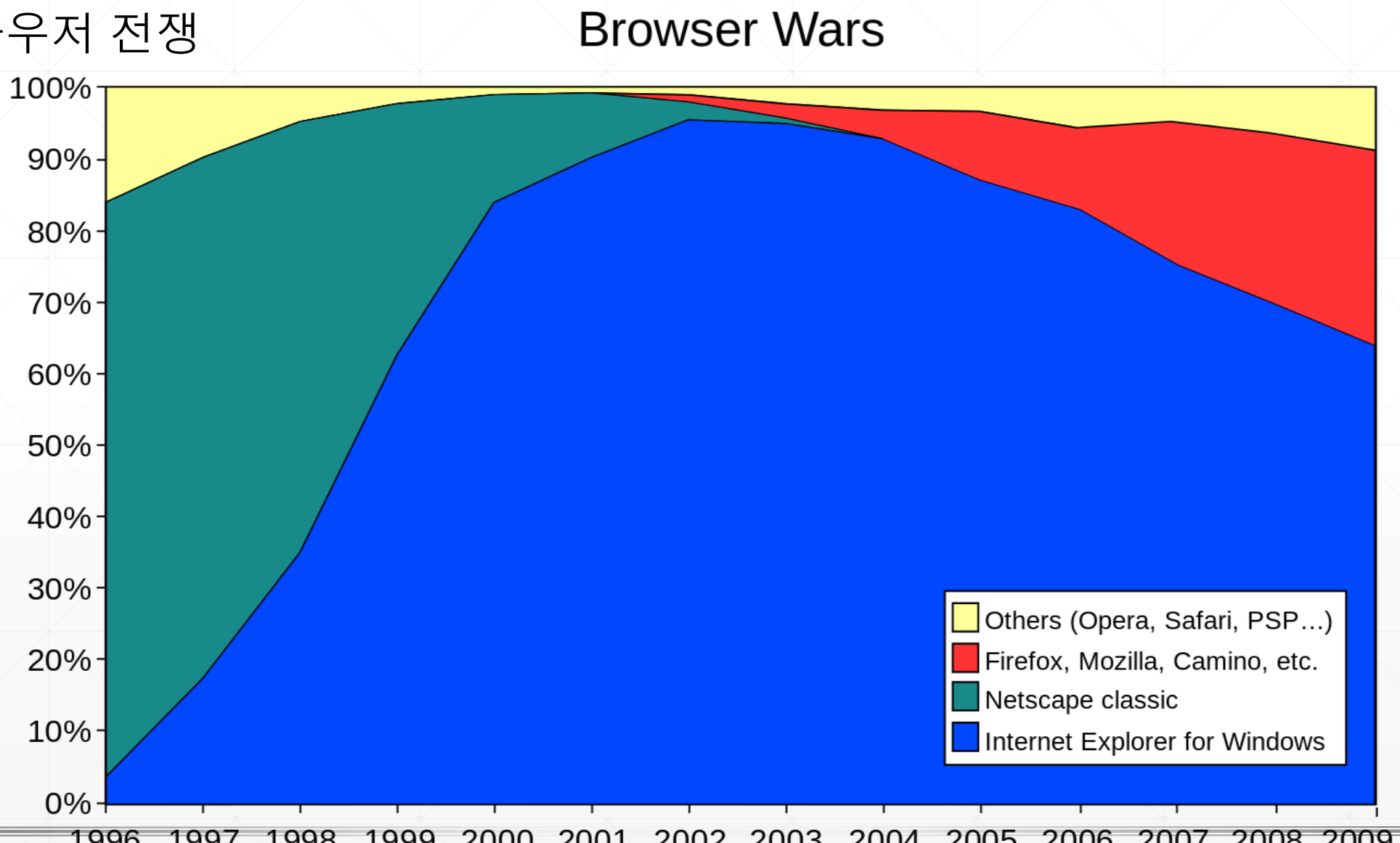
- 1995년, 자바스크립트가 Netscape에 탑재되기 시작
  - 새로운 사용자 경험과 함께 웹 시장 점유 시작

Microsoft

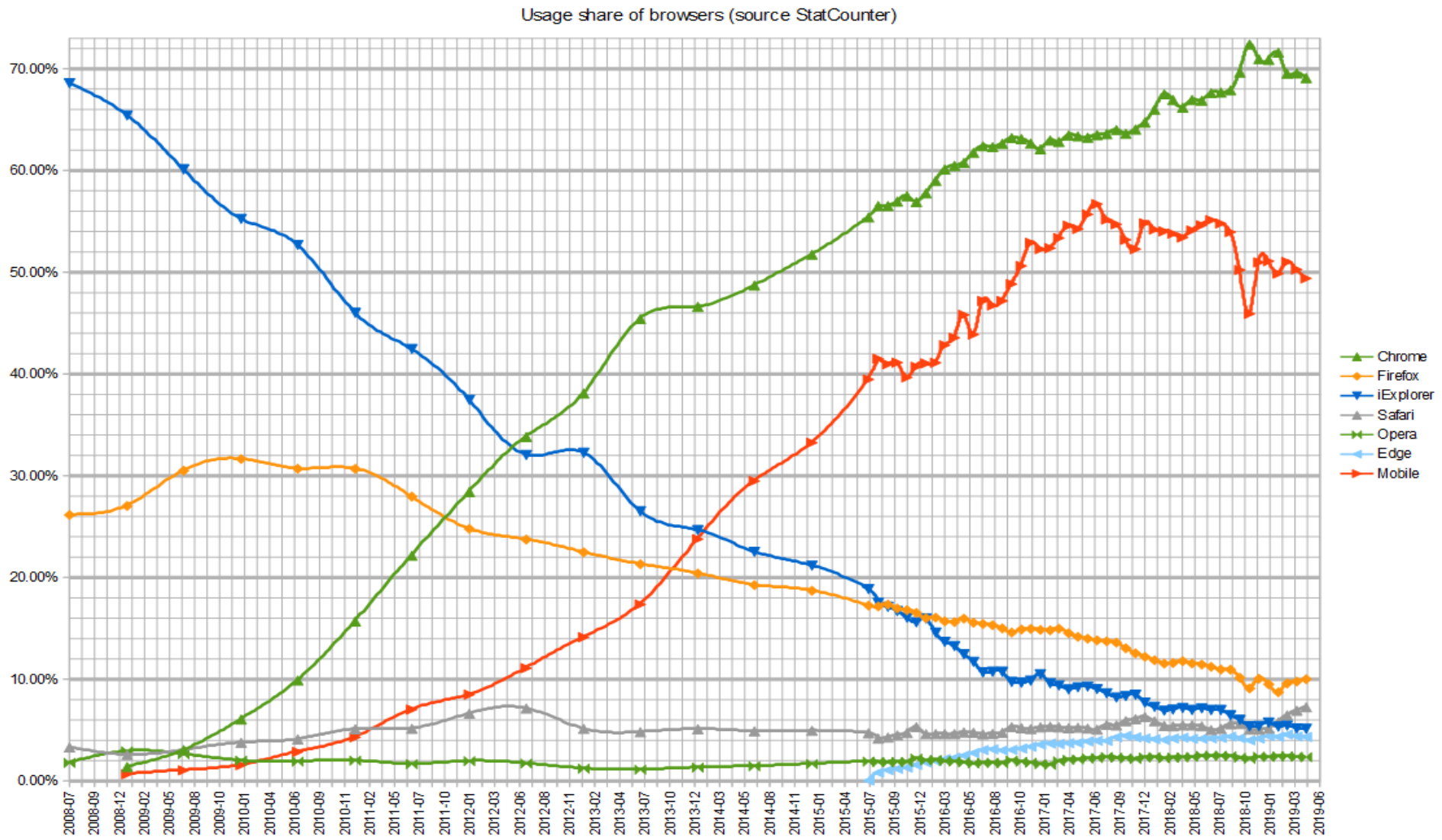
Internet Explorer

자바스크립트: 역사 (6/7)

- 브라우저 전쟁



자바스크립트: 역사 (6/7)


Usage share of browsers (source StatCounter)
70.00%
60.00%
50.00%
40.00%
30.00%
20.00%
10.00%
0.00%
2008-07
2008-09
2008-12
2009-02
2009-05
2009-08
2009-10
2010-01
2010-04
2010-06
2010-09
2010-11
2011-02
2011-05
2011-07
2011-10
2012-01
2012-03
2012-06
2012-08
2012-11
2013-02
2013-04
2013-07
2013-10
2013-12
2014-03
2014-05
2014-08
2014-11
2015-01
2015-04
2015-07
2015-09
2015-12
2016-03
2016-05
2016-08
2016-10
2017-01
2017-04
2017-06
2017-09
2017-12
2018-02
2018-05
2018-07
2018-10
2019-01
2019-03
2019-06
Chrome
Firefox
iExplorer
Safari
Opera
Edge
Mobile

자바스크립트: 역사 (6/7)

- 브라우저 전쟁

자바스크립트: 역사 (7/7)

- Javascript and Jscript (from Microsoft)
  - Java 상표권 이슈로 JScript로 명명
  - Javascript와 유사한 기능을 위한 별도 구현체
  - MS 인터넷 익스플로러 3.0부터 탑재되기 시작

- ECMAScript
  - ECMA-262 specification
    - Javascript 표준화를 위한 언어 specification
    - ECMA: European Computer Manufacturers Association
    - ECMAScript 5 (2009) / ECMAScript 6 (2015)

자바스크립트: 인기!

- 초기
  - Java의 아류작정도로 여겨짐
  - 디자이너, 스크립터, 비전문가용이라는 인식

- 현재
  - Front-end 뿐만 아니라 back-end까지 담당
  - 가장 많이 사용되는 프로그래밍 언어 중 하나로 발전

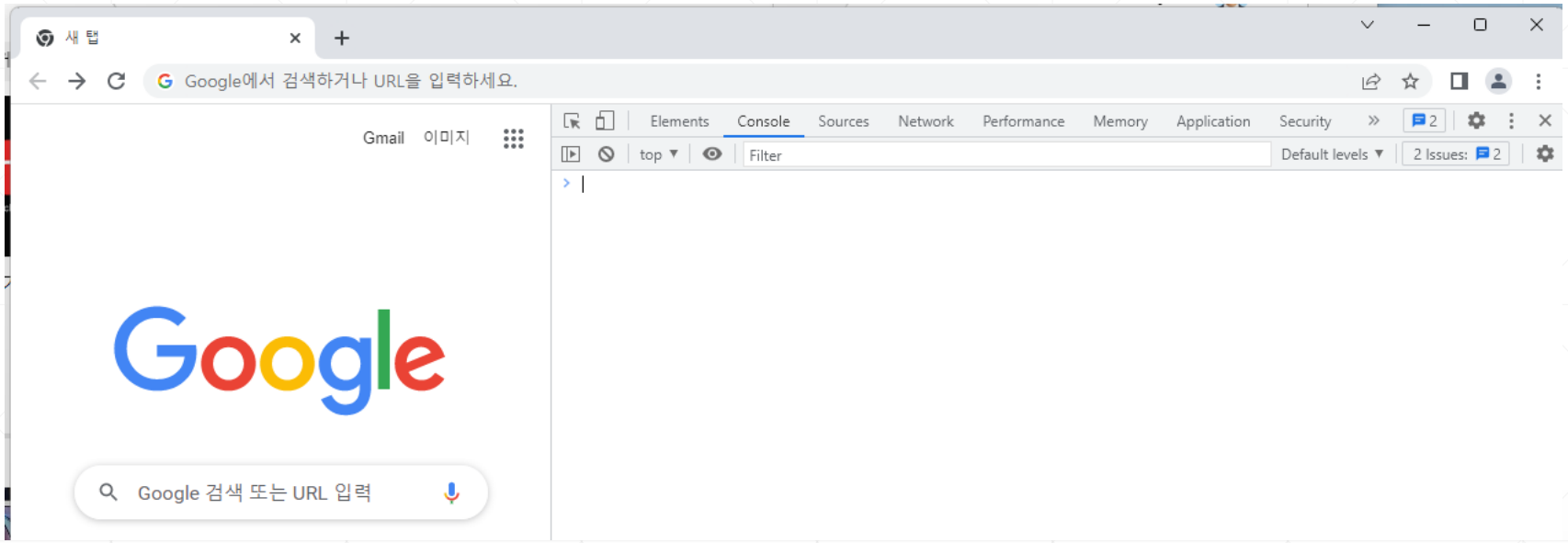
자바스크립트: 개발환경

- 스크립트 작성을 위한 에디터
  - Visual studio code 등
  - 크롬 개발자 도구 내 콘솔 등

- 스크립트 해석을 위한 인터프리터
  - 웹 브라우저 (웹 클라이언트)
  - Node.js (서버 측 javascript runtime)

자바스크립트: 개발 방식 1

- 개발자도구 – 콘솔



- 실습)
  - 아래 코드 입력 및 실행 결과 확인

```
console.log("안녕하세요?")
alert('웹 프로그래밍')
```

자바스크립트: 개발 방식 2

- HTML 문서 내부에 스크립트 작성
  - CSS3의 inline, internal 스타일시트와 유사
  - 문서 내부에 위치하며 브라우저에 의하여 해석됨

- HTML 문서 외부에 스크립트 작성
  - CSS3의 external 스타일시트와 유사
  - 별도 파일 형태로 위치하며 HTML에서 링크하여 사용
  - 해당 파일에 정의되어 있는 스크립트들이 해석됨

자바스크립트: 내부 스크립트

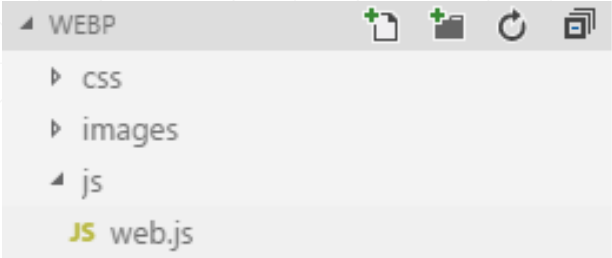
- 자바스크립트 코드는 <script></script> 요소안에 위치
- <head></head> 또는 <body></body> 내부에 위치 가능
- 예제)

```
<head>
<script>
        document.writeln('hello world!');
</script>
</head>
```

```
<body>
<script>
        document.writeln('hello world!');
</script>
</body>
```

자바스크립트: 외부 스크립트 (1/2)

- 별도의 자바스크립트 파일 생성
  - 별도 css 파일, 이미지/동영상 파일 생성과 동일
  - 예) js 폴더 내 web.js라는 이름의 별도 스크립트 파일 생성
  - 웹페이지의 구조와 로직의 분리 가능



- 스크립트 파일 로딩
  - 외부 CSS 파일 로딩과 유사한 방법 사용
  - 절대 및 상대 경로를 활용하여 스크립트 파일의 위치 지정

```
<head>
        <script src="js/web.js"> </script>
</head>
```

자바스크립트: 외부 스크립트 (2/2)

- 자바스크립트 파일 내용

index.html × JS web.js ×

js > JS web.js

```
1   document.writeln("Hello, World!")
2
```

  - 라이브 서버를 통하여 결과 확인!

- 외부 서버의 자바스크립트 파일도 로딩하여 사용 가능
  - 예) http://ixlab.seoultech.ac.kr/web.js

위 js 파일 링크는 2026년도에는
동작하지 않음

JS – 기본 연산자

JS 개요
객체 & 배열
함수
조건문 & 반복문

자바스크립트 기본 특성

- 간단한 문법 + 유연한 코딩 방법
  - 초기 진입장벽은 상대적으로 낮으나 깊이 들어갈수록 복잡!

- 언어적 특징
  - C/자바 계열과 유사한 문법을 가지고 있음
    - 괄호로 구분하는 블럭, 세미콜론으로 줄이 끝남을 알리는 것, 변수 쓰는 법, 연산자 사용법 등
  - 동적 바인딩 + 약타입 (타입 검사가 엄격하지 않음)
  - 이벤트 기반 프로그래밍
  - 객체기반/함수형 프로그래밍

문장 표현

- 기본적으로 Java/C의 문법과 유사
- Instruction들은 statement (문장)이라 불리며 세미콜론(;)으로 구분됨
  - 사실, 반드시 붙여야 되는 것은 아님
  - 그럼에도 불구하고 문장의 종료 시 세미콜론 할당을 권장함

- 주석처리
  - C++ 등의 주석처리와 동일

```
<script>
    // 나의 첫 주석!
    document.write('Hello, world!');
    /* 웹 프로그래밍
       자바스크립트
       HTML+CSS
       Node.js
    */
</script>
```

변수 (1/3)

- C/Java와는 달리 변수의 타입을 명시하지 않음
  - 타입: int, float, double 등
  - 값이 할당될 때 동적으로 내부적인 타입이 결정됨
    - typeof() 함수등으로 변수의 타입 확인 가능

데이터 타입	변수 값	비고
Number	정수, 실수 등 숫자 값을 가짐	숫자 (Number)와 문자열 (String) 타입간에는 숫자 값에 대해 자동 형변환을 제공함
String	연속된 글자들로 이루어진 문자열 (공백도 가능함). 문자열의 시작과 끝은 작은 따옴표 (') 혹은 겹따옴표 (")로 지정	
Boolean	true 혹은 false	조건식에서 사용
Undefined	undefined 만 가능	변수에 값이 할당되지 않음
Null	null 만 가능	"존재하지 않음"을 나타냄
Object	객체	객체, 배열 등

변수 (2/3)

- 변수의 이름
  - 유일한 이름을 가지도록 명명해야 함
  - 문자, 밑줄(_), 달러($), 숫자를 포함할 수 있음
  - 단, 시작은 문자, 밑줄(_), 달러($) 표기만 가능 (숫자 불가)
  - 대소문자 구별
  - 예약된 자바스크립트 키워드(if, for 등)는 사용 불가
  - camelCase로 표현
    - myScore, stuNumber, univCnt 등등
  - 의미있는 이름을 사용하도록 권장
    - aaa, bbb, abc123 지양

변수 (3/3)

- 변수의 유효범위(scope)에 따라 아래 키워드로 선언
  - var: 함수 스코프를 가지는 지역 변수 또는 전역 변수 선언
  - let: 블록 스코프를 가지는 지역 변수 선언
  - const: 블록 스코프를 가지는 상수 형태 선언

- 예제)

```
var myVar = "webp_var";
let myLet = 123456789;
const myConst = true;

console.log(myVar);
console.log(myLet);
console.log(myConst);

console.log("myVar: " + typeof (myVar));
console.log("myLet: " + typeof (myLet));
console.log("myConst: " + typeof (myConst));
```

연산자

- 산술 연산자: 사칙연산 + 나머지 연산자
- 문자열 연산자: 문자열 연결
- 증감 연산자: 값의 증가 또는 감소와 관련된 연산
  - ++, --
- 대입 연산자: 변수에 값을 할당하기 위한 연산자
- 비교 연산자: 피연산자간의 비교 연산
  - >, <, >=, <=, ==, !=, ===
- 논리 연산자: 피연산자간의 논리 연산
  - &&, ||, !

산술/문자열 연산자

- 사칙연산 + 나머지 연산
  - 연산 우선 순위는 Java와 동일
  - 문자열로 나타낸 숫자들끼리 사칙연산도 가능
  - 단, 문자열로 나타낸 숫자간의 덧셈 연산은 불가능!
- 문자열의 경우 덧셈 연산을 통해 문자열 연결 가능

```
console.log(2000+18);
console.log(2000-18);
console.log(2000*18);
console.log(2000/18);
console.log(2000%18);

console.log("2000"+"18"); // 문자열간 덧셈으로 적용됨
console.log("2000"-"18");
console.log("2000"*"18");
console.log("2000"/"18");
console.log("2000"%"18");
```

증감 / 대입 연산자 (1/2)

- 증감 연산자 (++, --)
  - 1을 더하거나 빼기 위한 연산자
  - 실행 순서에 따라 전위증감연산자와 후위증감연산자로 구분

```
var num = 1;
console.log(num++); // num의 값을 출력한 후 num + 1 (후위증감)
console.log(++num); // num + 1을 수행한 후 num의 값 출력 (전위증감)

console.log(--num); // num - 1을 수행한 후 num의 값 출력 (전위증감)
console.log(num--); // num의 값을 출력한 후 num -1 (후위증감)
console.log(num);
```

- 대입/복합 대입 연산자
  - 기본 대입 연산자 (=): 변수에 값을 할당
  - 복합 대입 연산자 (+=, -=, *=, /=): 산술연산과 대입연산의 축약형

증감 / 대입 연산자 (2/2)

대입 + 산술 연산 표현	복합 대입 연산 표현
a = a + 1	a += 1
a = a – 1	a -= 1
a = a * 1	a *= 1
a = a / 1	a /= 1

- 복합 대입 연산자 예제)

```
var num = 1;

num = num + 1;
console.log(num);
num += 10; // num = num + 10;
console.log(num);
num = num / 2;
console.log(num);
num /= 2; // num = num / 2;
console.log(num);
```

비교 연산자 (1/7)

- 피연산자간의 대소부등관계 검사를 위한 연산자
  - 비교 연산의 결과에 따라 참(true)/거짓(false)을 결과로 반환

비교 연산자	의미
>, >=	좌측이 우측보다 큼 (크거나 같음)
<, <=	좌측이 우측보다 작음 (작거나 같음)
==, !=	좌측과 우측의 값이 동일함 (동일하지 않음)
===, !==	좌측과 우측의 값과 타입이 동일함 (동일하지 않음)

  - 문자열간의 비교 연산은 alphabetical order에 따름
    - “car”가 “ar”보다 큰 문자열, “2”가 “12”보다 큰 문자열
  - 다른 타입 간의 비교 연산
    - 규칙에 따라 타입 변환 후 비교
    - 숫자로 변환 불가능한 값은 NaN(Not A Number)로 변환됨

비교 연산자 (2/7)

- 비교 연산 예제)

```
console.log(2 < 12);
console.log(2 < "12"); // 문자열의 형변환
console.log(2 < "Jinwoo"); // "Jinwoo" --> NaN
console.log(2 > "Jinwoo");
console.log(2 == "Jinwoo");
console.log("2" < "12"); // 문자열간의 비교
console.log("2" > "12");
console.log("2" == "12");
```

- "==, !=" 와 "===, !==" 의 차이점?
  - ==, !=는 값을 비교
    - 자동 형변환 수행 후 값을 비교
  - ===. !==는 값과 타입까지 비교

비교 연산자 (3/7)

- 자동 형변환에 의한 == 연산 예제)

```
console.log(0 == 0);
console.log(0 == "0");
console.log(0 == "000");
console.log(0 == []); // 빈 배열
console.log(0 == false);
console.log(0 == ""); // 빈 문자열
console.log(0 == undefined);
console.log(0 == null);
```

  - == 연산을 수행하기 전 자동 형변환 규칙에 의하여 우측 피연산자들의 타입을 변경한 후 최종 값을 바탕으로 비교 연산 수행
  - 피연산자의 타입에 따라 자동 형변환 규칙이 다름

비교 연산자 (4/7)

■ 상등 (==) 연산 시 자동 형변환 규칙

		Operand B					
		Undefined	Null	Number	String	Boolean	Object
Operand A	Undefined	`true`	`true`	`false`	`false`	`false`	`false`
	Null	`true`	`true`	`false`	`false`	`false`	`false`
	Number	`false`	`false`	`A === B`	`A === ToNumber(B)`	`A === ToNumber(B)`	`A == ToPrimitive(B)`
	String	`false`	`false`	`ToNumber(A) === B`	`A === B`	`ToNumber(A) === ToNumber(B)`	`A == ToPrimitive(B)`
	Boolean	`false`	`false`	`ToNumber(A) === B`	`ToNumber(A) === ToNumber(B)`	`A === B`	`ToNumber(A) == ToPrimitive(B)`
	Object	`false`	`false`	`ToPrimitive(A) == B`	`ToPrimitive(A) == B`	`ToPrimitive(A) == ToNumber(B)`	`A === B`

- ToNumber(): Number type으로 변환하는 추상 연산자
- ToPrimitive(): 반대쪽 피연산자의 타입으로 변환 시도하는 추상 연산자

비교 연산자 (5/7)

- 형변환 테이블 (1/2)

Original Value	Converted to Number	Converted to String	Converted to Boolean
false	0	"false"	false
true	1	"true"	true
0	0	"0"	false
1	1	"1"	true
"0"	0	"0"	true
"000"	0	"000"	true
"1"	1	"1"	true
NaN	NaN	"NaN"	false
Infinity	Infinity	"Infinity"	true
-Infinity	-Infinity	"-Infinity"	true

비교 연산자 (6/7)

■ 형변환 테이블 (2/2)

Original Value	Converted to Number	Converted to String	Converted to Boolean
""	0	""	false
"20"	20	"20"	true
"twenty"	NaN	"twenty"	true
[ ]	0	""	true
[20]	20	"20"	true
[10,20]	NaN	"10,20"	true
["twenty"]	NaN	"twenty"	true
["ten","twenty"]	NaN	"ten,twenty"	true
function(){}	NaN	"function(){}"	true
{ }	NaN	"[object Object]"	true
null	0	"null"	false
undefined	NaN	"undefined"	false

비교 연산자 (7/7)

- 타입과 함께 비교를 위하여 ===, !== 연산 활용
  - 보다 정확하고 엄격한 값 비교
  - 피연산자들의 타입이 다를 경우 무조건 false
  - 타입이 같을 경우 값 비교

```
console.log(0 === 0);
console.log(0 === "0");
console.log(0 === "000");
console.log(0 === []); // 빈 매열
console.log(0 === false);
console.log(0 === ""); // 빈 문자열
console.log(0 === undefined);
console.log(0 === null);
```

논리 연산자 (1/2)

- 논리합 연산자 (A || B)
  - A 또는 B가 참(true)일 경우 참(true)

- 논리곱 연산자 (A && B)
  - A와 B가 모두 참(true)이어야 참(true)
  - 둘 중 하나라도 거짓(false)이면 거짓(false)

- 부정 연산자 (!A)
  - A가 거짓(false)이면 참(true)
  - A가 참(true)이면 거짓(false)

논리 연산자 (2/2)

- 예제)

```
var myAge = 35;
var yourAge = 25;

console.log(myAge >= 30 || yourAge >= 30);
console.log(myAge >= 30 && yourAge >= 30);

console.log(!(myAge >= 30 || yourAge >= 30));
console.log(!(myAge >= 30 && yourAge >= 30));
```

JS 개요
JS – 기본 연산자

객체 & 배열

함수
조건문 & 반복문

객체 (1/4)

- 정보의 저장/표현을 위한 구조
  - 중괄호({})를 이용하여 객체 선언
  - 속성과 메소드로 구성되어 있음

- 속성
  - 키:값의 형태로 구성되는 객체의 성질
  - 속성의 값으로 어떠한 형태의 데이터가 와도 상관없다!

- 메소드
  - 키:함수의 형태로 구성되는 객체의 동작/기능
  - 속성의 값 자리에 함수가 오는 경우

객체 (2/4)

객체 선언

```
var jinwoo = {
    firstName: "jinwoo",
    lastName: "jeong",
    eNum: "02621",
    age: 39,
    eat:function(food){alert(food + " 맛있겠네~");}
}
```

- firstName 속성: “jinwoo” (문자열)
- lastName 속성: “jeong” (문자열)
- eNum 속성: “02621” (문자열)
- age 속성: 39 (숫자)
- eat 메소드 (함수)

객체 (3/4)

객체의 속성 접근

- 마침표(.)를 이용한 접근
  - 객체.속성명 형태로 접근
- 대괄호([])를 이용한 접근
  - 객체["속성명"] 형태로 접근

```
> jinwoo.age
<· 39
> jinwoo.eNum
<· '02621'
> jinwoo.firstName
<· 'jinwoo'
> jinwoo.lastName
<· 'jeong'
```

```
> jinwoo['age']
<· 39
> jinwoo['eNum']
<· '02621'
> jinwoo['firstName']
<· 'jinwoo'
> jinwoo['lastName']
<· 'jeong'
```

객체 (4/4)

- 객체안의 객체

```
var jinwoo = {
    firstName: "jinwoo",
    lastName: "jeong",
    eNum: "02621",
    age: 39,
    eat:function(food){alert(food + " 맛있겠네~");},
    class:{
        name:"webp",
        where:"frontier"
    }
}
```

- 접근: jinwoo.class.name // “webp”

배열 (1/2)

- 다양한 종류의 다수의 데이터를 저장하기 위한 구조
  - 파이썬의 리스트와 유사
  - 배열의 각 요소가 동일한 데이터 타입을 가지지 않아도 된다
  - 배열의 크기가 언제라도 증가, 감소가 가능

- 선언 방법
  - 배열의 이름
  - 배열 표기
  - 배열 내용

```
my_list = [1, 2, 3, 4, 5]
```

배열 (2/2)

■ 배열 선언 예제

```
var arr1 = [ ] ;
var arr2 = [1, 2, 3];
var arr3 = ["Life", "is", "too", "short"];
var arr4 = [1, 2,"Life","is"];
var arr5 = [1, 2, ["Life", "is"]];
```

- 빈 배열
- 숫자로 이루어진 배열
- 문자열로 이루어진 배열
- 숫자와 문자열이 혼합된 배열
- 배열을 요소로 가지는 배열
- …

배열의 접근 (1/2)

- 배열의 길이
  - Length 속성 이용 (arr3.length = 4)

- 대괄호와 인덱스를 이용하여 요소에 접근
  - 인덱스: 배열에서 각 요소의 위치를 나타내는 숫자

0 1 2

var arr2 = [1, 2, 3];

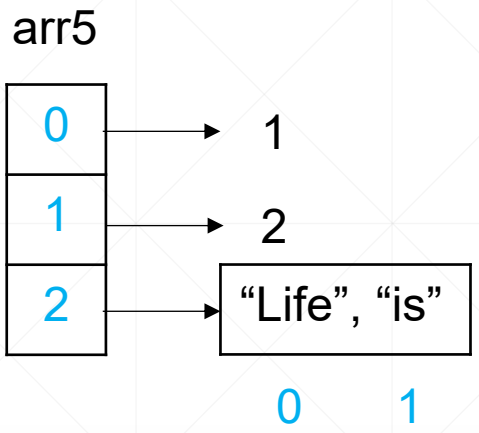
  - arr2[0]; // 1
  - arr2[1]; // 2
  - arr2[3]; // undefined
  - arr2[2] = 4; // 4

배열의 접근 (2/2)

■ 2차원 요소에 대한 접근

0 1 2

var arr5 = [1, 2, ["Life", "is"]];



- arr5[2]; // [“Life”, “is”]
- arr5[2][0]; // “Life”
- arr5[2][1]; // “is”
- arr5[2][2]; // undefined
- arr5[2][2] = “Too short”; // “Too Short”

배열 기본 연산 (1/3)

- 파이썬 리스트와 유사한 기능 보유

Array methods	의미
arr.push(x)	배열의 맨 마지막에 x를 추가
arr.pop()	배열의 맨 마지막 요소를 제거 제거 된 요소는 결과값으로 반환 됨
arr.concat(X)	주어진 배열 X와 연결하여 하나의 배열을 생성 후 반환
arr.shift	배열의 맨 앞 요소를 제거 제거 된 요소는 결과값으로 반환 됨
arr.unshift(x)	배열의 앞 앞에 x를 추가
arr.indexOf(x)	배열에서 첫번째로 발견되는 x의 인덱스를 반환
arr.sort()	배열을 정렬 (정렬 기준: 문자열 순서)
arr.reverse()	배열을 역순으로 재배치
arr.slice(start, upto)	start위치부터 upto 이전까지 추출하여 새로운 배열 생성
arr.splice(start, num [,x,…])	start위치에서 num만큼 삭제 후 주어진 x들을 삽입 제거 된 요소는 결과값으로 반환 됨

배열 기본 연산 (2/3)

- Slice 연산
  - 배열의 일부를 추출하여 새로운 배열 생성
  - 원본 배열에는 영향을 미치지 않음
  - arr.slice(start, upto); // start에서 시작하여 upto 이전까지 추출

var arr = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9];

  - arr.slice(4, 7); // [4, 5, 6]
  - arr.slice(4, 7).slice(1, 2); // [5]
  - var newArr = arr.slice(0, 5); // newArr = [0, 1, 2, 3, 4]

배열 기본 연산 (3/3)

Splice 연산

- 배열의 일부를 삭제 (삭제 후 주어진 값들로 교체)
- 삭제 된 요소들은 결과값으로 반환되며 원본 배열에 영향을 미침
- arr.splice(start, num [,x,…]); // start에서 num 만큼 삭제 후 주어진 x들로 값 교체

var arr = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9];

- arr.splice(0,5); // arr = [5, 6, 7, 8, 9] (0 위치에서 5개를 삭제)
- var newArr = arr.splice(0,3); // arr = [8, 9], newArr = [5, 6, 7]
- arr.splice(0, 0, 5, 6, 7); // arr = [5, 6, 7, 8, 9]
  - 0의 위치에서 삭제하지 않고 7, 8, 9로 교체
- arr.splice(2, 2, 1, 2); // arr = [5, 6, 1, 2, 9]
  - 2의 위치에서 2개 삭제 후 1, 2로 교체

JS 개요
JS – 기본 연산자
객체 & 배열

함수

조건문 & 반복문

함수 (1/10)

- Python/Java의 메소드와 유사한 개념
- 입력 인자를 바탕으로 가공을 통하여 결과값을 반환
  - 입력, 결과 값 반환은 필수적인 부분은 아님

- 함수 선언식 (declaration)
  - 함수의 이름
  - 파라미터의 리스트
  - 함수의 몸체(body)

```
function sum(num1, num2) {

        return num1 + num2;

}
```

함수 (2/10)

함수 표현식 (expression)

- 변수에 함수를 선언하여 할당하는 방식

```
var myFunc = function sum(num1, num2) {
        return num1 + num2;
}
```

- 함수가 할당된 변수의 이름으로 호출 가능
  - 따라서, 함수 표현식에서는 함수의 이름을 별도로 정의할 필요 없음
  - 익명 함수 (anonymous function)라고 불림

```
var myFunc = function (num1, num2) {
        return num1 + num2;
}
```

함수 (3/10)

- 파라미터 초기값 (default parameter)
  - 기본적으로 파라미터의 초기값은 undefined를 지님
  - 초기값 설정이 필요할 경우 사용
  - 함수 이름(파라미터 = 초기값) 형태로 설정 가능

```
function multiply(a, b = 10) {
        return a * b;
}
```

  - multiply() 함수의 두 번째 인자가 주어지지 않을 경우 b를 10으로 설정
  - multiply() 함수의 두 번째 인자가 주어질 경우 해당 인자를 사용
  - multiply(10), multiply(10,15)의 값에 차이가 나는가?

함수 (4/10)

Arrow 함수

- 일반적인 함수의 축약형
- Java/Kotlin의 람다 함수와 유사
- 입력 파라미터와 함수의 몸체만을 기술하여 함수 정의

기본 형식: (입력 파라미터) => { 함수의 몸체; }

일반적인 익명 함수

```
function (num1, num2) {
        return num1 + num2;
}
```

Arrow 함수

```
(num1, num2) => {return num1 + num2;}
```

함수 (5/10)

객체의 메소드

- 객체의 속성 값이 함수인 경우 메소드라고 불림
- 객체.메소드(인자) 형태로 호출

```
var jinwoo = {
    firstName: "jinwoo",
    lastName: "jeong",
    eNum: "02621",
    age: 39,
    eat:function(food){alert(food + " 맛있겠네~");}
}
```

- jinwoo 객체의 eat 메소드
  - alert함수를 호출하는 익명 함수로 이루어짐
  - Arrow 함수로 변경할 수 있을까?

함수 (6/10)

- 파라미터 형태의 함수 전달 (콜백: callbacks)
  - 함수를 변수에 (또는 객체의 속성에) 할당할 수 있을 뿐만 아니라, 함수의 인자로도 전달할 수 있음!
  - 전달받은 함수 인자를 실행시킬 수도 있음!

- 예) 두 개의 숫자 인자와 서로 다른 기능을 함수를 입력받는 함수의 구현
  - myFunc(num1, num2, callback)
    - num1, num2: 첫/두 번째 숫자 인자
    - callback: 사칙연산을 수행하기 위한 4개의 서로 다른 함수

함수 (7/10)

- 사칙연산을 위한 각각의 콜백 함수

```
var add = function (x, y){return x+y;}

var subt = function (x, y){return x-y;}

var multi = (x, y) => {return x*y;}

var div = (x, y) => {return x/y;}
```

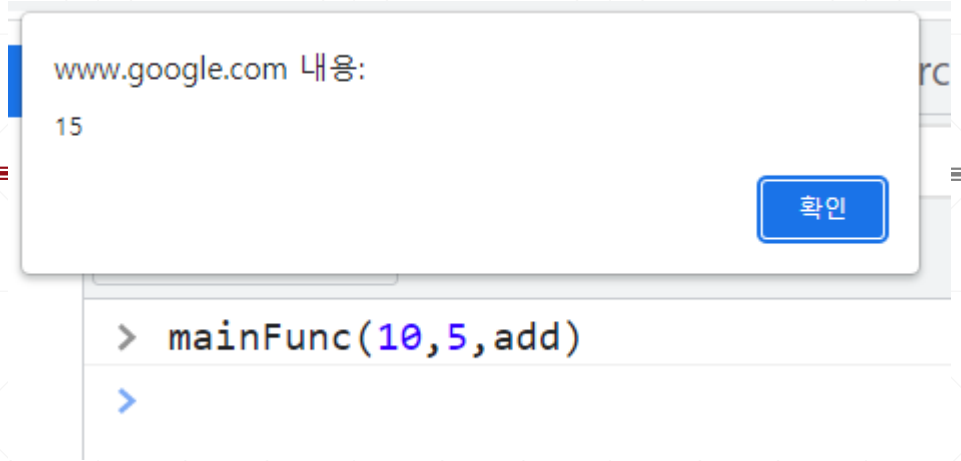
- 숫자와 사칙연산 콜백을 인자로 받는 메인 함수

```
var mainFunc = function(num1, num2, callback){

        alert(callback(num1,num2));

}
```

함수 (8/10)



- mainFunc() 실행 예제)

```
var mainFunc = function(num1, num2, callback){

        alert(callback(num1,num2));

}
```

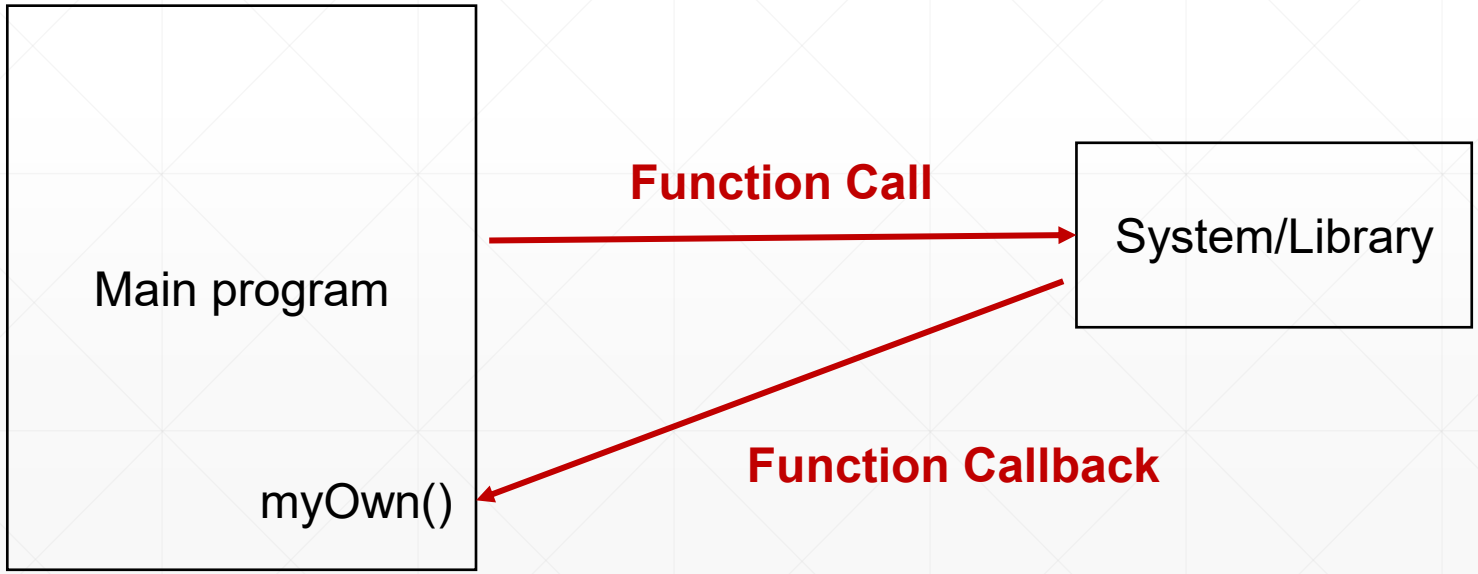
10 5

```
add = function (x, y) {
    return x+y;
}
```

- 인자로서 전달된 함수를 그대로 실행!
  - Callback 함수

함수 (9/10)

- Call-back?
  - 일반적인 함수 호출의 흐름: 시스템/라이브러리가 제공하는 함수 호출
  - 시스템/라이브러리에서 메인 프로그램이 제공하는 함수 역호출



함수 (10/10)

Callback의 또다른 예

```
> var count = 0;
<· undefined
> setInterval(()=>{console.log(++count);},1000);
```

우리가 제공하는 Arrow 함수

- setInterval(code, delay)
  - 주어진 code를 delay만큼의 간격으로 호출
- 위의 예: 우리가 제공하는 익명 함수를 1초 간격으로 호출하여 기능 수행

JS 개요
JS – 기본 연산자
객체 & 배열
함수

조건문 & 반복문

조건문

- C/Java/Python의 조건문과 매우 유사
  - if-else 구문
  - 3항 연산자 (if else의 축약형)
  - switch 구문

- If … else if … else 구문
  - 조건문이 참인 경우 해당 구문을 실행
  - 거짓에 해당하는 값
    - false
    - undefined/null/NaN
    - 0
    - empty string (“”)

조건문 – if

- 기본 조건문 형식

```
if (condition) {
    statement_1;
} else {
    statement_2;
}
```

Condition이 참일 경우 실행

Condition이 거짓일 경우 실행

- 3항 연산자
  - condition ? A : B
  - Condition이 참이면 A, 거짓이면 B

조건문 – if

- If-else 구문과 3항 연산자

```
> var score = 95;
<· undefined
> var grade = "F";
<· undefined
> if (score > 90){
      grade = "A";
  } else {
      grade = "C";
  }
<· "A"
> grade
<· "A"
```

If – else 구문

```
> var score = 70;
<· undefined
> var grade = (score > 90) ? "A" : "C";
<· undefined
> grade
<· "C"
```

3항 연산자 구문

조건문 – if

- 확장된 조건문 형식

```
if (condition_1) {
    statement_1;
} else if (condition_2) {
    statement_2;
} else if (condition_n) {
    statement_n;
} else {
    statement_last;
}
```

배타적 조건

조건문 – switch

- If … else if … else 구문과 유사
  - 배타적 조건문 (expression과 매치되는 case의 구문만을 실행)
- Switch 구문 형식

```
switch (expression) {
  case label_1:
    statements_1
    [break;]

  case label_2:
    statements_2
    [break;]
    ...
  default:
    statements_def
    [break;]
}
```

검사 대상

배타적 조건

매칭되는 case가 없을 경우

조건문 – switch

- Switch 예제)

```
var fruittype = prompt('어떤 과일을 좋아하니?');
switch (fruittype) {
    case '오렌지':
    console.log('오렌지는 2000원~');

    case '사과':
    console.log('사과는 200원~');
    break;

    case '바나나':
    console.log('바나나는 20원~');
    break;

    default:
    console.log('미안... ' + fruittype + '은 다 떨어졌어...');
}
```

Break 구문을 삭제하면?

반복문

- 반복문 또한 C/Java/Python의 반복문과 매우 유사!
  - For
  - While
  - Do… while
  - break/continue
  - For … in
  - For … of

반복문 – for/while

- For 구문
  - 종료 조건이 만족할 때까지 반복 수행

```
for ([초기값]; [종료 조건]; [증감문]){

        statements
}
```

- While 구문
  - 종료 조건이 만족할 때까지 반복 수행

```
while (종료 조건){

    statements

}
```

반복문 – for/while

- 예제) 0부터 9까지 순서대로 출력하기 위한 프로그램

For 버전

```
for (i=0;i<10;i++){
    alert(i);
}
```

While 버전

```
var cnt = 0;
while(cnt < 10){
    alert(cnt);
    cnt++;
}
```

- 예제) H1 태그 글꼴의 크기를 1~10em까지 증가시키며 출력하기 위한 프로그램

```
for (i=1;i<11;i++){
    document.write("<h1 style='font-size:"+i+"em'>Hey~</h1>");
}
```

  - While 버전은?

반복문 – Do … While

- 최초 한 번의 구문 실행 후 반복 조건 검사

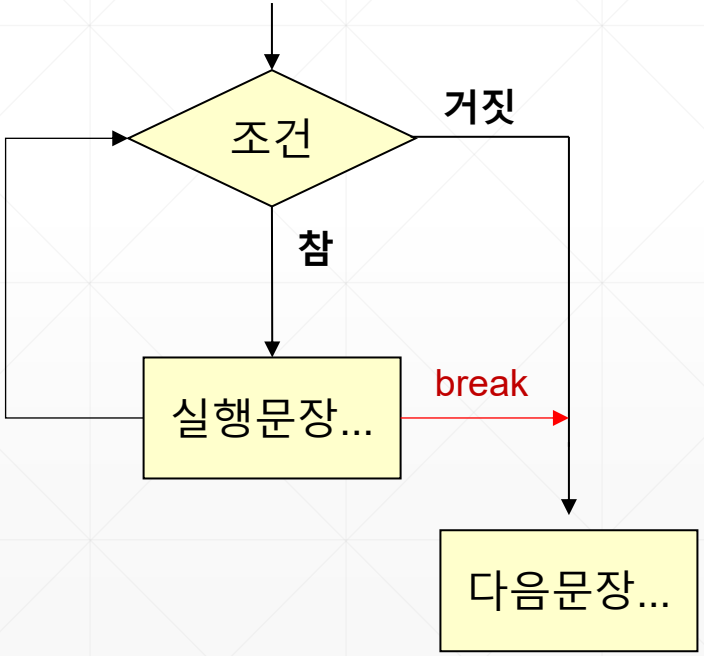
```
do {
    statements
}
while (종료조건);
```

- 예) 로그인 예제

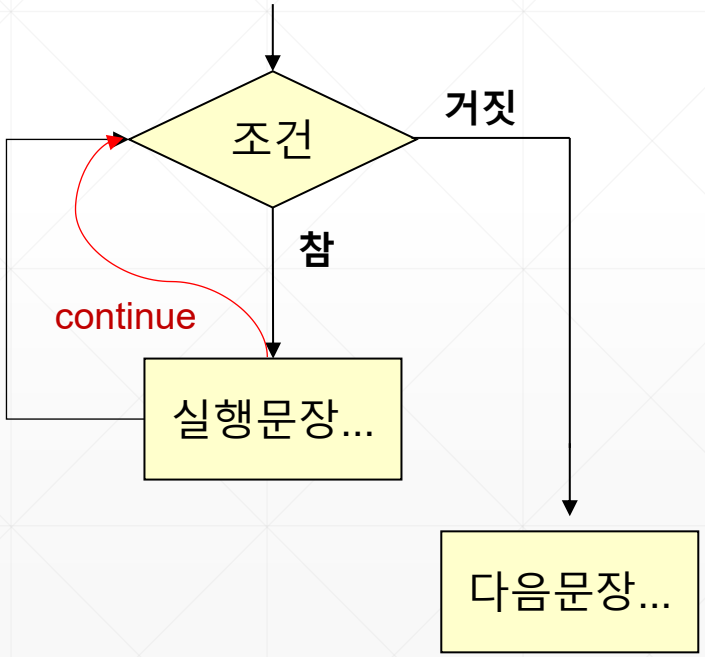
```
var input, password="pass";
do {
    input = prompt("암호를 입력하세요");
} while(input !== password);
alert('로그인 완료!');
```

반복문 – break & continue

- 반복문 수행 흐름
  - Break: 현재 반복문을 중단시킴
  - Continue: 현재 반복문의 다음 반복 수행을 위하여 점프



Break 구문의 흐름도



Continue 구문의 흐름도

반복문 – break & continue

■ 예제)

```
for(i=1;i<=10;i++){
    if(i%5==0) break;
    console.log(i);
}
```

```
1
2
3
4
```

Break!

```
for(i=1;i<=10;i++){
    if(i%5==0) continue;
    console.log(i);
}
```

```
1
2
3
4
6
7
8
9
```

Next!

반복문 - 객체/배열의 요소 순회 (1/6)

- 객체의 속성이나 배열 등의 요소를 순회하기 위한 도구
  - 반복문 (for, for…in, for…of)
  - 배열의 메소드 (forEach)

- for 구문을 이용한 배열 요소 순회
  - 배열의 길이를 획득한 후 인덱스를 이용하여 요소 순회

```
var colors = ['red', 'green', 'blue'];

for (var i = 0; i < colors.length; i++) {
    console.log(colors[i]);
}
```

반복문 - 객체/배열의 요소 순회 (2/6)

- Array.forEach 구문

```
arr.forEach(function callback(요소의 값 [, 인덱스 [, 배열]]) {
    //your iterator
}[, thisArg]);
```

- 배열의 각 요소에 대하여 넘겨진 콜백 함수를 실행
- 요소의 값: 현재 반복중인 요소의 값
- 요소의 인덱스: 현재 반복중인 요소의 위치(인덱스)
- 배열: 현재 반복중인 배열

반복문 - 객체/배열의 요소 순회 (3/6)

■ Array.forEach 구문 (계속)

```
var arr = ['a', 'b', 'c'];

for(i=0;i<arr.length;i++){
    console.log(arr[i]);
}

arr.forEach(function(element) {
    console.log(element);
});
```

Arrow 함수 버전

```
arr.forEach(element=>{
    console.log(element);
});
```

➢ 추가 인자를 사용한 요소 순회

```
arr.forEach((element,index,array)=>{

    console.log("값: "+element+" 인덱스: "+index+" 전체 배열: "+array);

});
```

반복문 - 객체/배열의 요소 순회 (4/6)

- For … of 구문을 이용한 배열의 요소 순회
  - Python의 for … in 구문과 유사

```
for (variable of array) {
    statements
}
```

현재 순회중인 요소가 variable에 할당됨

  - 예제)

```
var arr = ['a', 'b', 'c'];

for(i=0;i<arr.length;i++){
    console.log(arr[i]);
}

for (ch of arr){
    console.log(ch);
}
```

기본 **for** 구문

for … of구문

반복문 - 객체/배열의 요소 순회 (5/6)

■ For … in 을 쓰면 안되나요?

```
var arr = ['a', 'b', 'c'];

for(i=0;i<arr.length;i++){
    console.log(arr[i]);
}
for (ch in arr){
    console.log(ch);
}
```

실행 결과 →

```
a
b
c
0
1
2
```

- For…in 구문은 객체의 속성의 이름을 순회
- For…of 구문은 배열(iterable object)내 요소들의 값을 순회
- 배열에 속성을 임의로 추가하면?
  - arr.dummy=“dummy”;

반복문 - 객체/배열의 요소 순회 (6/6)

- For … in은 객체의 속성 순회에 사용

```
var jinwoo = {
    firstName: "jinwoo",
    lastName: "jeong",
    eNum: "02621",
    age: 39,
     class:{
         name:"webp",
         where:"frontier"
}
}
for (prop in jinwoo){
    console.log(prop);
    console.log(jinwoo[prop]);
}
```

```
firstName
jinwoo
lastName
jeong
eNum
F00483
age
35
class
▶ {under: "webp", grad: "tech"}
```